

Reviewer Pack — Minimal Order of Totally Ramified Extensions vs DPLL Refutat...

Entry 7ef8e0eceeef · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 06:55:51 UTC

Minimal Order of Totally Ramified Extensions vs DPLL Refutation Depth

Entry ID: 7ef8e0eceeef **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 06:55:51 UTC

1. Conjecture

Field A (mathematical branch): Number Theory (Totally Ramified Extensions) **Field B** (complexity object): Complexity Theory: DPLL Refutation Complexity

Statement:

[‘For a given satisfiable CNF formula F , the minimal order k of a totally ramified extension K/Q containing Q with finite Galois group isomorphic to $\mathbb{Z}/k\mathbb{Z}$, satisfies $O(k) \leq \log_2(t(F))$ for some constant c .’, ‘Where $t(F)$ is the minimal depth of the DPLL refutation tree of F and $\log_2$ is the binary logarithm.’, ‘Equivalently, for any satisfiable CNF formula F with n variables, if K/Q is a totally ramified extension of degree k containing Q such that $[Q:K] = k$ and $G(K/Q) \cong \mathbb{Z}/k\mathbb{Z}$, then $O(k) \leq \log_2(t^*(F))$.’]

Rationale (proposer’s reasoning):

[‘Totally ramified extensions can be used to create fields with a simple Galois group structure, which might capture certain properties of the refutation process in DPLL algorithms.’, ‘The minimal order of such extensions could potentially reflect the complexity of the formula’s truth table or its structure, providing insights into the refutation process.’, ‘This conjecture bridges algebraic number theory with computational complexity by proposing a quantitative relationship between the structure of a field and the complexity of solving a SAT instance.’]

Taxonomy category: TROPICAL_FOURIER_ANALYSIS (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d9fdd4f40138085c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient between $\log_2(k)$ and $\log_2(t(F))$ across at least 30 random seeds is ≥ 0.8 , where k is the minimal order of a totally ramified extension and $t(F)$ is the DPLL refutation depth.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal order of totally ramified extensions" AND "DPLL refutation depth" - "Galois group isomorphic to Z/kZ " AND "satisfiable CNF formula DPLL" - "order k totally ramified extension" related to "DPLL tree depth"

Top relevant hits considered: - [<http://arxiv.org/abs/cs/0608100v1>] Similarity of Semantic Relations - [<http://arxiv.org/abs/0810.1207v1>] A Layered Grammar Model: Using Tree-Adjoining Grammars to Build a Common Syntactic Kernel for Related Dialects - [<http://arxiv.org/abs/1310.8154v3>] Characteristic cohomology of the infinitesimal period relation

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from itertools import combinations, product

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for i in range(1, n + 1):
            clause = [random.choice([i, -i]) for _ in range(random.randint(2, 3))]
            clauses.append(clause)
        return clauses

    def dpll(cnf, assignment={}):
        if not cnf:
            return True
        unit_clauses = [c[0] for c in cnf if len(c) == 1]
        if unit_clauses:
            var = unit_clauses[0]
            if -var in assignment and assignment[-var]:
                return False
            assignment[var] = True
            cnf = [[x for x in c if x != var and x != -var] for c in cnf]
            return dpll(cnf, assignment)
```

```

pure_literals = {}
for literal in set([abs(x) for c in cnf for x in c]):
    pos_count = sum(1 for c in cnf if literal in c)
    neg_count = sum(1 for c in cnf if -literal in c)
    if pos_count == 0:
        pure_literals[literal] = False
    elif neg_count == 0:
        pure_literals[literal] = True
if pure_literals:
    var, value = next((k, v) for k, v in pure_literals.items())
    assignment[var] = value
    cnf = [[x for x in c if x != var and x != -var] for c in cnf]
    return dpll(cnf, assignment)
p_var = next(var for var in range(1, len(assignment) + 2) if var not in assignment)
return dpll(cnf, assignment | {p_var: True}) or dpll(cnf, assignment | {p_var: False})

def dpll_refutation_depth(cnf):
    depth = [0] * (len(cnf) + 1)
    stack = [(cnf, {})]
    while stack:
        cnf, assignment = stack.pop()
        if not cnf:
            return max(depth)
        unit_clauses = [c[0] for c in cnf if len(c) == 1]
        if unit_clauses:
            var = unit_clauses[0]
            if -var in assignment and assignment[-var]:
                continue
            assignment[var] = True
            depth[len(assignment)] += 1
            stack.append([(x for x in c if x != var and x != -var) for c in cnf], assignment)
        else:
            p_var = next(var for var in range(1, len(assignment) + 2) if var not in assignment)
            depth[len(assignment)] += 1
            stack.append([c[:] for c in cnf], assignment | {p_var: True})
            stack.append([c[:] for c in cnf], assignment | {p_var: False})
    return max(depth)

def minimal_totally_ramified_extension_order(n):
    # This is a placeholder function. Replace with actual computation.
    return random.randint(2, n)

n = random.randint(5, 40)
cnf = generate_cnf(n)
k = minimal_totally_ramified_extension_order(n)
t_star = dpll_refutation_depth(cnf)

if t_star == float('inf'):
    return {
        "metric_name": "log2(k) vs log2(t*)",
        "metric_value": None,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "DPLL refutation depth is infinite"
    }
}

```

```

if k == 0:
    return {
        "metric_name": "log2(k) vs log2(t*)",
        "metric_value": None,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "Minimal order of extension is zero"
    }

log2_k = math.log2(k)
log2_t_star = math.log2(t_star)

return {
    "metric_name": "log2(k) vs log2(t*)",
    "metric_value": log2_k,
    "instances_tested": 1,
    "conjecture_holds": log2_k <= log2_t_star,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or list(range(2, 30)) # Default to first 29 primes

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    total_metric_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None)
    mean_metric_value = total_metric_value / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_92b44a0f.py", line 120, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_92b44a0f.py", line 83, in run_trial

```

```
t_star = dpll_refutation_depth(cnf)
~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_92b44a0f.py", line 71, in dpll_refutati
depth[len(assignment)] += 1
~~~~~
```

IndexError: list index out of range

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing any data, which means that the Pearson correlation coefficient could not be calculated. Therefore, the conjecture cannot be supported or falsified based on this test. | next: Re-run the test to ensure it completes without crashing and produces the necessary data for calculating the Pearson correlation coefficient.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15621
2	preregistration	ollama_remote	glm4:latest	0	0	6092
3	novelty	ollama_remote	glm4:latest	0	0	4799
4	novelty	ollama_remote	glm4:latest	0	0	8834
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13146
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12955
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9095
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15380
9	judge	ollama_remote	glm4:latest	0	0	16138

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 102059 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/7ef8e0eceeef.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/7ef8e0eceeef.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/7ef8e0eceeef.tar.gz (if generated)